RANS-PINNs Implementation Plan

From Laminar Aneurysm PINN to Turbulent High-Re Flow

Concept, Theory, and Step-by-Step Code Modifications | PhysicsNeMo Sym

> *How to use this document: Part A explains the theory and design decisions. Part B is the exact, ordered sequence of edits to apply to your existing Python script. Follow Part B top to bottom in your code file.*

Document Map

Part	Contents
Part A	Concept overview, governing equations, constants, boundary condition theory, training strategy, validation, limitations
Part B	Ordered code changes — exact edits to apply to your script, in the sequence you should make them
Appendix	Open items requiring your input before this is fully runnable

PART A — Concept & Theory

A.1 Problem Setup & Overview

Parameter	Value
Description	Extend your laminar PINN (Navier-Stokes only) to RANS k-epsilon for turbulent flow
Your current Re	150 (laminar — confirmed from your hardcoded parameter table)
Target Re for this upgrade	10,000 - 150,000 (e.g. automotive exhaust, full throttle)
Approach	Reynolds-Averaged Navier-Stokes + k-epsilon closure embedded as additional PDE loss terms
Framework	PhysicsNeMo Sym
Flow Regime	Steady-state, incompressible, turbulent

> *Why this matters: at Re=150 your existing script is valid as-is. The changes below are only needed if/when you move to a higher-Re geometry where turbulence cannot be ignored.*

A.2 Network Architecture Changes

The network must predict 6 output fields instead of 4. A larger network capacity is recommended to capture turbulent multi-scale features.

Parameter	Laminar (Your Original)	RANS-PINNs (Target)
Input Keys	x, y, z	x, y, z (unchanged)
Output Keys	u, v, w, p	u, v, w, p, k, ep
Hidden Layers	as set in conf/config.yaml	8 (recommended)
Neurons per Layer	as set in conf/config.yaml	512 (recommended)
Activation Function	as configured	SiLU or tanh

A.3 Governing PDE Equations

Continuity (unchanged)

du/dx + dv/dy + dw/dz = 0 — from NavierStokes built-in class

Momentum (RANS form)

rho * (u . grad) u = -grad(p) + div((nu + nu_t) * grad(u))

This requires an effective viscosity nu_eff = nu + nu_t coupling the mean flow to the turbulence fields. See Appendix item 1 — this coupling is not a one-line change.

k Transport Equation

u . grad(k) = div((nu + nu_t/sigma_k) * grad(k)) + P_k - epsilon

Epsilon Transport Equation

u . grad(ep) = div((nu + nu_t/sigma_e) * grad(ep)) + C1*(ep/k)*P_k - C2*(ep^2/k)

Turbulent Viscosity

nu_t = C_mu * k^2 / epsilon

Production Term

P_k = nu_t * S^2, where S^2 = 2*Sij*Sij, Sij = 0.5*(dui/dxj + duj/dxi)

Expanded: S^2 = 2(du/dx)^2 + 2(dv/dy)^2 + 2(dw/dz)^2 + (du/dy+dv/dx)^2 + (du/dz+dw/dx)^2 + (dv/dz+dw/dy)^2

A.4 Standard k-Epsilon Constants

Source: Launder & Sharma (1974) — Standard k-epsilon model

Constant	Value	Description
C_mu	0.09	Turbulent viscosity coefficient
C1_epsilon	1.44	Epsilon production coefficient
C2_epsilon	1.92	Epsilon destruction coefficient
sigma_k	1.0	Turbulent Prandtl number for k
sigma_epsilon	1.3	Turbulent Prandtl number for epsilon

A.5 Boundary Condition Theory

Inlet

Variable	Formula	Notes
u, v, w	Parabolic / plug profile	Same as your existing circular_parabola() function
k	k_inlet = 1.5 * (U_ref * I)^2	I = 0.05 (5% turbulence intensity, typical for pipe inlet)
ep	ep = C_mu^0.75 * k^1.5 / (0.07 * D_h)	D_h = hydraulic diameter of inlet

Outlet

Variable	Condition
p	p = 0 (unchanged)
k, ep	Zero gradient — do NOT constrain

Wall (No-Slip)

Variable	Condition	Notes
u, v, w	0	Unchanged
k	0	Zero turbulent KE at wall
ep	Wall function (high-Re) or low-Re formula	Use wall functions for Re > 30,000 to avoid resolving viscous sublayer

A.6 Numerical Stabilization

Issue	Fix
epsilon -> 0 (division by zero)	ep_safe = Max(ep, 1e-8)
Negative k	k_safe = Max(k, 1e-8)
Excessive eddy viscosity	nu_t = Min(nu_t, 100*nu)
Gradient explosion	Gradient clip norm = 1.0
Early loss imbalance	Start k/ep lambda_weighting at 0.01–0.1

A.7 Training Strategy — 3 Phases

Phase	Iterations	Active Losses	k/ep Weight	Goal
1. NS Pretrain	50,000	continuity, momentum only	0.0	Establish mean flow before introducing turbulence
2. Soft Coupling	50,000	All losses	0.01	Introduce k-ep without destabilizing NS
3. Full RANS	100,000	All losses	0.1	Converge full coupled system

Optimizer: Adam, cosine annealing LR from 1e-3 to 1e-6, gradient clip norm 1.0, total ~200,000 iterations.

A.8 Recommended Batch Sizes

Region	Batch Size	Change from Laminar?
Inlet	256	No
Outlet	256	No
No-slip (Wall)	2048	Increase — wall-dominated turbulence
Interior	4096	Increase — turbulent PDE residual density

Integral Continuity	512	No

A.9 Validation Metrics

Category	Metric	Target
Velocity / Pressure	u, v, w, p vs OpenFOAM	L2 error < 5%
Turbulence fields	k, ep, nu_t	Qualitative match to OpenFOAM k-ep run
Wall	y+	< 1 (low-Re) or 30-100 (wall functions)
Turbulence intensity	sqrt(2k/3)/U_ref	Compare to inlet spec (5%)

A.10 Known Limitations

- RANS-PINNs practically validated up to Re ~ 10^5; higher Re needs domain decomposition.
- Separated / recirculating flows are harder to converge — needs denser interior sampling there.
- Training takes roughly 10-20x longer than your current laminar setup.
- RANS gives a time-averaged result only — cannot capture transient exhaust pulsation (would need URANS or LES for that).

PART B — Ordered Code Changes

Apply these edits to your script in this exact order. Each step says where in the file it goes relative to your existing code.

Step 1 — Add imports for custom PDE class

Location: top of file, with your other imports.

```
from physicsnemo.sym.eq.pde import PDE
from sympy import Symbol, Function, Min, Max
```

Step 2 — Add the KEpsilon PDE class

Location: after your imports, before the @physicsnemo.sym.main decorator / run() function.

```
class KEpsilon(PDE):
    def __init__(self, nu, rho=1.0):
        x, y, z = Symbol("x"), Symbol("y"), Symbol("z")
        u, v, w = Function("u")(x,y,z), Function("v")(x,y,z), Function("w")(x,y,z)
        k, ep = Function("k")(x,y,z), Function("ep")(x,y,z)

        C_mu, C1, C2 = 0.09, 1.44, 1.92
        sigma_k, sigma_e = 1.0, 1.3

        k_safe = Max(k, 1e-8)
        ep_safe = Max(ep, 1e-8)
        nu_t = Min(C_mu * k_safe**2 / ep_safe, 100*nu)

        S2 = (2*u.diff(x)**2 + 2*v.diff(y)**2 + 2*w.diff(z)**2
              + (u.diff(y)+v.diff(x))**2 + (u.diff(z)+w.diff(x))**2
              + (v.diff(z)+w.diff(y))**2)
        P_k = nu_t * S2

        k_eqn = (u*k.diff(x) + v*k.diff(y) + w*k.diff(z)
                 - ((nu+nu_t/sigma_k)*k.diff(x)).diff(x)
                 - ((nu+nu_t/sigma_k)*k.diff(y)).diff(y)
                 - ((nu+nu_t/sigma_k)*k.diff(z)).diff(z)
                 - P_k + ep_safe)

        ep_eqn = (u*ep.diff(x) + v*ep.diff(y) + w*ep.diff(z)
                  - ((nu+nu_t/sigma_e)*ep.diff(x)).diff(x)
                  - ((nu+nu_t/sigma_e)*ep.diff(y)).diff(y)
                  - ((nu+nu_t/sigma_e)*ep.diff(z)).diff(z)
                  - C1*(ep_safe/k_safe)*P_k + C2*(ep_safe**2/k_safe))

        self.equations = {"k_equation": k_eqn, "ep_equation": ep_eqn}
```

Step 3 — Update network output keys

Location: inside run(), where flow_net = instantiate_arch(...) is defined. Replace the output_keys line.

```
flow_net = instantiate_arch(
    input_keys=[Key("x"), Key("y"), Key("z")],
    output_keys=[Key("u"), Key("v"), Key("w"), Key("p"), Key("k"), Key("ep")],  #
k, ep added
    cfg=cfg.arch.fully_connected,
)
```

Step 4 — Instantiate KEpsilon and update node list

Location: right after your existing 'ns = NavierStokes(...)' and 'normal_dot_vel = NormalDotVec(...)' lines, replace the nodes = (...) block.

```
ns = NavierStokes(nu=nu * scale, rho=1.0, dim=3, time=False)
ke = KEpsilon(nu=nu * scale, rho=1.0)   # NEW
normal_dot_vel = NormalDotVec(["u", "v", "w"])
flow_net = instantiate_arch(...)  # from Step 3

nodes = (
    ns.make_nodes()
    + ke.make_nodes()              # NEW
    + normal_dot_vel.make_nodes()
    + [flow_net.make_node(name="flow_network")]
)
```

Step 5 — Add k, ep targets to the inlet constraint

Location: where you compute u, v, w via circular_parabola(...) and build the inlet constraint, add the k_inlet / ep_inlet calculation just before it, then extend outvar.

```
# turbulence inlet values (NEW)
I = 0.05
D_h = 2 * inlet_radius
k_inlet = 1.5 * (inlet_vel * I)**2
ep_inlet = (0.09**0.75) * k_inlet**1.5 / (0.07 * D_h)

inlet = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=inlet_mesh,
    outvar={"u": u, "v": v, "w": w, "k": k_inlet, "ep": ep_inlet},  # k, ep added
    batch_size=cfg.batch_size.inlet,
)
```

Step 6 — Add k = 0 to the no-slip constraint

Location: your existing no_slip = PointwiseBoundaryConstraint(...) block.

```
no_slip = PointwiseBoundaryConstraint(
    nodes=nodes,
    geometry=noslip_mesh,
    outvar={"u": 0, "v": 0, "w": 0, "k": 0},  # k added
```

```
    batch_size=cfg.batch_size.no_slip,
)
```

Step 7 — Add k_equation, ep_equation to the interior constraint

Location: your existing interior = PointwiseInteriorConstraint(...) block. Also add lambda_weighting (not present in your original script).

```
interior = PointwiseInteriorConstraint(
    nodes=nodes,
    geometry=interior_mesh,
    outvar={
        "continuity": 0, "momentum_x": 0, "momentum_y": 0, "momentum_z": 0,
        "k_equation": 0, "ep_equation": 0,   # added
    },
    lambda_weighting={                        # NEW block
        "continuity": 1.0, "momentum_x": 1.0, "momentum_y": 1.0, "momentum_z":
1.0,
        "k_equation": 0.01, "ep_equation": 0.01,
    },
    batch_size=cfg.batch_size.interior,
)
```

Step 8 — Outlet constraint: no change required

Your existing outlet block (outvar={"p": 0}) stays exactly as-is. Do not add k or ep here.

Step 9 — Update batch sizes in conf/config.yaml

Location: your Hydra config file (not the Python script).

```
batch_size:
  inlet: 256
  outlet: 256
  no_slip: 2048      # increased from original
  interior: 4096     # increased from original
  integral_continuity: 512
```

Step 10 — Staged training (run separately, not a single code block)

This is a training-script / config change, not a single in-place edit:

1. Phase 1: temporarily remove k_equation/ep_equation from interior outvar, train ~50,000 iterations to get a stable mean flow.
2. Phase 2: add k_equation/ep_equation back with lambda_weighting = 0.01, train ~50,000 more iterations (resume from checkpoint).
3. Phase 3: raise lambda_weighting to 0.1, train remaining ~100,000 iterations to convergence.

Summary of Files Touched

File	Changes
your_script.py	Steps 1-8: imports, KEpsilon class, output keys, node list, inlet/no-slip/interior constraints
conf/config.yaml	Step 9: batch sizes; Step 10: training iteration counts per phase

Appendix — Open Items Before This Is Fully Runnable

These are not optional polish — without them the RANS coupling is incomplete or your geometry constants are wrong.

#	Item	Why it matters	What's needed
1	NS momentum equation does not yet use nu_eff = nu + nu_t	Without this, k and ep are solved but don't actually feed back into the velocity field — the turbulence model isn't truly two-way coupled	Write a custom NavierStokes-style PDE class (similar structure to KEpsilon) that substitutes nu_eff in the momentum residual
2	inlet_normal flagged 'Verify from STL' in your parameter table	circular_parabola() depends on this for correct inlet velocity direction	Re-extract from your actual STL geometry, not reused from the original aneurysm mesh
3	outlet_area_raw flagged 'Provide actual value'	Used in outlet_radius and integral continuity constraints	Measure from your CAD/STL outlet face
4	Wall function implementation for Re > 30,000	Low-Re ep wall formula assumes near-wall mesh resolution PINNs don't naturally provide	Decide between resolving y+ < 1 (expensive) or implementing a wall-function boundary treatment